

Aflevering 2

Mikkel Moth Billing - 07/03-24

I denne opgave får vi noget data over hvordan en K^- partikel spredtes på protoner. Den data vi får givet burde kunne beskrives ved et polynomie, så vi tester hvilken grad der giver det bedste plot. Vores t data er cosinus til spredningsvinklen, og y dataen er antallet af tællinger.

Jeg har lavet et plot, med errorbars for datapunkterne, hvor afvigelsen er kvadratroden af tællertallet. Efter det har jeg lavet et for loop fra 0 til 5, så jeg kan lave polynomier fra 0'te til 5'te grad. Jeg har valgt at bruge `np.polyfit` til at fitte dataen. I polyfittet har jeg givet min usikkerhed, og valgt at den ikke skal skaleres. Så giver polyfit mig parameterne til polynomiet, og χ^2 . Så har jeg brugt `ss.chi2.sf` til at finde p-værdien, hvor frihedsgraderne er 10-antallet af parameter. Jeg har plottet fittene ved hjælp af `np.polyval` som tager parameterne og t som input.

Ud fra plottet, χ^2 og p, kan vi se at 0., 1. og 2. grads polynomierne ikke er gode til at fitte vores data. Man kan godt sige at 3. grad er tilstrækkeligt, da den har $\chi^2 = 2,850$ og $p = 0,827$, men 4. grads polynomiet har den højeste p værdi $p = 0.891$, og en $\chi^2 = 1.686$ der er lavere end 3. grad. For 5. grads polynomiet, falder χ^2 en lille smule, men p falder også. Hvilket vil sige at vi har for lidt frihedsgrader.

```
In [1]: import numpy as np
import matplotlib.pyplot as plt
import scipy.stats as ss
from scipy.optimize import curve_fit
plt.rc("axes", labelsz=16)
plt.rc("xtick", labelsz=16, top=True, direction="in")
plt.rc("ytick", labelsz=16, right=True, direction="in")
plt.rc("axes", titlesz=18)
plt.rc("legend", fontsize=10)
```

```
In [3]: t = np.array([-0.9, -0.7, -0.5, -0.3, -0.1, 0.1, 0.3, 0.5, 0.7, 0.9])
y = np.array([81, 50, 35, 27, 26, 60, 106, 189, 318, 520])
yerr = np.sqrt(y)
t_a = np.linspace(-1,1,100)

fig, ax = plt.subplots()
ax.errorbar(t,y, yerr, fmt="o", capsize=5, label="Data")

for n in range(6):
    params, chmin, _, _ = np.polyfit(t, y, n, full=True, w=1/yerr, cov="unscaled")
    p = ss.chi2.sf(chmin,9-n)
    ax.plot(t_a, np.polyval(params, t_a),
            label=f"{len(params)-1}. grad,  $\chi^2$={chmin[0]:.3f}, p={p[0]:.3f}")
    print('chi2:',chmin, ' ---> p:', p)

ax.legend()
ax.set_xlabel("Cosinus til spredningsvinklen")
ax.set_ylabel("Tællinger")
ax.grid('on')
ax.set_title("Polynomie fit")
plt.show()$ 
```

```
chi2: [833.54752248] ---> p: [1.27384331e-173]
chi2: [585.44869204] ---> p: [3.14134421e-121]
chi2: [36.40964776] ---> p: [6.06644656e-06]
chi2: [2.84989163] ---> p: [0.8274375]
chi2: [1.68602272] ---> p: [0.89065593]
chi2: [1.66264864] ---> p: [0.79749083]
```

Polynomie fit

